This handout includes space for every question that requires a written response. Please feel free to use it to hand write your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset LaTeX solutions.

1.a [6 points (Written)] No Shuffle Required

We have provided you with omniglot.py, which contains code for task construction and data loading. Recall that for training black-box meta-learners in the previous homework we needed to shuffle the query examples in each task. This is not necessary for training protonets. Explain why.

Answer: In short, the guiding principle is *permutation invariance* to ensure task generalizability. Shuffling of the *query* examples is not needed for *protonets* because they have no memory of the order of the *query* sequence–it is implicitly a sample permutation invariant architecture. However, shuffling of the *query* examples is necessary for *black-box* meta-learners because in practice they do see the *query* example ordering and thus the *black-box* network can learn to generate the exact same sequence of classes, essentially over-fitting. I.e. shuffling in the black-box case is necessary to ensure some degree of permutation invariance is achieved.

Additional Details:
Back-box Meta-learners: Recall in the case of the black-box meta-learner in Homework 2, we had to shuffle the order of examples in the query set, as otherwise the network can learn to output the same sequence of classes and achieve 100% accuracy, without actually learning to recognize the images. If we were to get 100% accuracy, we likely did not shuffle the query data correctly. In this case, the meta-learner architecture used a long-short term memory (LSTM) network, an advanced form of recurrent neural networks (RNN).

Protonetworks: Shuffling the *query* examples in each task is NOT necessary for protonets because protonets are purely feedforward networks–in this case, we are using a convolutional neural network (CNN) which has no sequence memory. Thus there is no risk of over-fitting to or even learning from the *query* example sequence ordering. The query examples are simply fed through the network and mapped to an embedding space that places query features close to (e.g. via Euclidean distance) the embeddings of similar features in the prototypical classification categories. Ergo, the learning approach is fully independent of *query* sequence ordering.

1.b [6 points (Coding)] Implement Step

Here is the plot of the validation query accuracy over the course of training:
(Hint: We should obtain a query accuracy on the validation split of at least 99%.)
And here is the benchmark figure from the Handout for comparison:

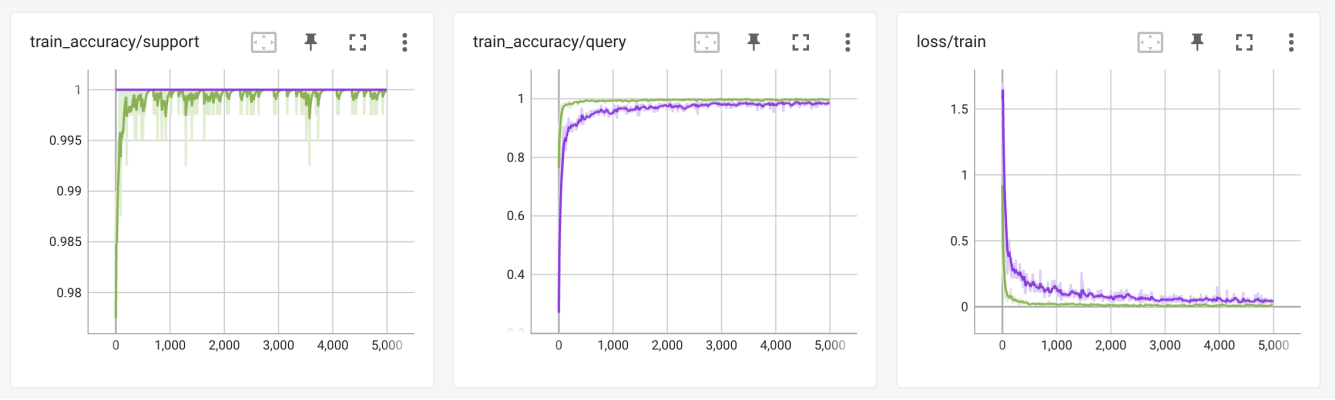


Figure 1b: Protonetwork training Support example accuracy (left), Query example accuracy (middle), and training Loss (right) curves as a function of epoch. Colors are: 5-way, 1-shot (purple; N=5, K=1, Q=15), and 5-way, 5-shot (green; N=5, K=5, Q=15). Smoothing was set to 75% to ease synoptic interpretation.

1.c [6 points (Written)] Evaluating Accuracy Metrics

(i) Is the model placing support examples of the same class close together in feature space or not? Support your answer by referring to specific accuracy metrics.

Answer: Yes. The CNN model is doing a good (>99%) job placing *support* examples of the same class close together in feature embedding space (see *Figure 1b* above, green trace, leftmost plot; *"training_accuracy/support"*).

Additional Details: Recalling that accuracy is defined by the class that maximizes the unnormalized log probabilities ($logits = -d^2$), while we would expect an accuracy of 100% in the case of N-way and 1-Shot, in the case of 5-Shot here, we should expect slightly <100% here which we've observed.

I.e. the distance between *support* examples of the same class and their prototypical class representations in the featured embedding space learned by the CNN is a nearly 0 Euclidean distance.

(ii) Is the model generalizing to new tasks? If not, is it overfitting or underfitting? Support your answer by referring to specific accuracy metrics.

Answer: Yes, the model is generalizing to new tasks, per the convergence to a consistently high *query* example accuracy (above *Figure 1b*, middle plot, green trace), as well as a classification loss converging appropriately toward 0 (right plot); achieving a near 0 loss even after just the first several hundred epochs.

1.d [6 points (Written)] 5-way 1-shot Comparison We will now compare different settings at training time. Train on 5-way 1-shot tasks with 15 query examples per task.

.

(i) Compare your two runs (5-way 1-shot training and 5-way 5-shot training) by assessing test performance on 5-way 1 shot tasks. To assess a trained model on test tasks, run:

```
        python protonet.py --test

    specifically in my case:
        log_dir=\
        "logs/protonet_successfull_way_5.support_1_and_5_query/\
        omniglot.way_5.support_1.query_15.lr_0.001.batch_size_16/"

        python protonet.py --test --num_way 5 --num_support 5 --checkpoint_step 1000 \
        --log_dir $log_dir
```

appropriately specifying log dir and checkpoint step. Submit a table of your results with 95% confidence intervals.

Answer: Table of 4 different comparisons:

N-Way K-Support	**Epoch**	**Mean**	**95% Conf**
a1: 5-Way 5-Support	0	0.777	0.006
a2: 5-Way 5-Support	1000	0.987	0.002
a3: 5-Way 5-Support	**4900**	**0.994**	**0.001**
b1: 5-Way 1-Support	0	0.585	0.009
b2: 5-Way 1-Support	1000	0.952	0.004
b3: 5-Way 1-Support	**4900**	**0.978**	**0.003**

(Note: All rows in the above table each included 600 test tasks in the accuracy calculations.)

Visually, looking at *Figure 1b* shown previously, it's clear at least qualitatively that the *5-way (N), 5-support (K)* trained model (green trace) is more general, less overfit and higher performance than the *5-way, 1-support* model (black trace).

Quantitatively, looking at the table above and specifically for the 4900th iteration (model near final trained convergent state), it's clear that there is a measurable difference between the *5-way 5-support* model performance accuracy (99.4%) and the *5-way 1-support* model accuracy (97.8%)). While the 1.6% difference between these two models seems trivial, the *5-way 5-support* model is a more accurate solution with a narrower 95% confidence interval (per table above), and we'll explain this finding more quantitatively and algorithmically in **(iii)**.

(ii) How did you choose which checkpoint to use for testing for each model?

Answer: I chose the final checkpoint available (4900th iteration) as the most informative to use as it represents the near final state of the trained models. Two other checkpoints (0 and 1000 iterations) are also listed to help paint the picture. Metrics from the 4900th and all three iterations reviewed match the visual interpretation of *Figure 1b* shown previously–differences described qualitatively in **(i)** and quantitatively in **(iii)**.

(iii) Is there a significant difference in the test performance on 5-way 1-shot tasks? Explain this by referring to the protonets algorithm.

Answer: Yes, there is a statistically significant difference, corroborated by tIt's instructive to display copies of Equation 1 and 5 from the Handout here:

$$c_n = \frac{1}{K} \sum_{(x,y) \in \mathcal{D}_i^{\text{tr}} : y = n} f_\theta(x), \tag{1}$$

$$\mathcal{J}(\theta) \approx \frac{1}{B} \sum_{i=1}^{B} \left[\frac{1}{NQ} \sum_{(x^{\text{ts}}, y^{\text{ts}}) \in \mathcal{D}_i^{\text{ts}}} -\log p_\theta(y = y^{\text{ts}} \mid x^{\text{ts}}) \right]. \tag{5}$$

Detailed answer:
From Equation 5, the training loss $J(\theta)$ which we are optimizing is explicitly inversely proportional to the number of "ways" (N in equation 5), and implicitly to the number of "supports" (K in equation 1). If we consider $\hat{c}_n$ to approximate the best representation of the n-th class prototype c_n, it's clear that as the number of supports (K) increases, $\hat{c}_n$, converges to the maximum likelihood estimate of the best mean c_n.

In quantitative terms, the probability of conducting additional task tests and observing miss classifications outside of 99.4% +/- 0.1% is $\leq$5%. For the *5-way, 5-support* model, we can confidently expect 95% of additional task tests with correct classifications to result in an accuracy in the range of [99.3% to 99.5%], compared to the *5-way, 1-support* model's non-overlapping and lesser expected accuracy range [97.5% to 98.1]%. Ergo, the *5-way, 5-support* is a statistically verifiable, significantly better model that the *5-way, 1-support* model.

2.a [12 points (Coding)] Implement Inner Loop

See maml.py.

STEPS in my coding solution:
Step 0: Adaptation to Post-Adaptation: Train for num_inner_steps + 1 more.
Post Adaptation: One more forward pass to get the post-adaptation accuracy
Step 1: Forward Pass
Step 2: Calculate Loss and Accuracy each training step
Step 3: Adaptation: Use autograd.grad for Backward Pass during Training

Hints that I leveraged or developed:
a. Train for only num_inner_steps as that is the adaptation part and then after that do one more forward pass to get the post_adapt accuracy.
b. The simplest way to implement inner loop involves using autograd.grad.
b.1. The train boolean variable is needed only in the autograd call. It is important, while in train mode (bool variable in the inner loop), to call autograd with create_graph, as the graph will be used for the 2nd order derivatives in the backward in the outer loop.
b.2. Calculate all the gradients at once by passing list(parameters.values()) into autograd.grad
b.3. Make sure you're updating the gradients in the inner loop as that is the adaptation part. During training you're teaching the network how to adapt better (outer loop) but it still has to adapt (inner loop).
c. Use util.score for the accuracies as it makes the code easier to debug.
d. The cross entropy function expects raw logits.

Slack conversation:
https://xcs330-scpd.slack.com/archives/C06DS0V2Z8F/p1706891800705289

2.b [12 points (Coding)] Implement Outer Loop

See maml.py.

STEPS in my coding solution:
Step 1: Adapt the Parameters (inner loop) using Support Images only. The support data is used to adapt the parameters to ϕ^L.
Step 2: Optimize θ using Query Images To optimize θ in the outer loop, we use the same loss function (6) applied on the adapted parameters and the query data.
Step 3: Accuracy using provided score() against our unnormalized log probabilities (logits)
Step 4: Calculate Loss on the Query data

Hints that I leveraged or developed:
- Use the autograd in the inner loop and the backward in the outer loop
- To understand how to use the Boolean train argument of MAML, outer step, read the documentation for the create_graph argument of autograd.grad.
- The outer loop parameters are only updated at training time, but this is already implemented for you in the train method. You don't need to update the outer (shared) parameters anywhere in the code blocks.

Assessing my implementation of vanilla MAML on 5-way 1-shot Omniglot:
(Hint: you should obtain a query accuracy on the validation split of at least 96%.)
The post-adaptation Query accuracy on the validation data is 98% at the final epoch. And the Gradescope Autograder tests all passed. Thus implementation is performant.

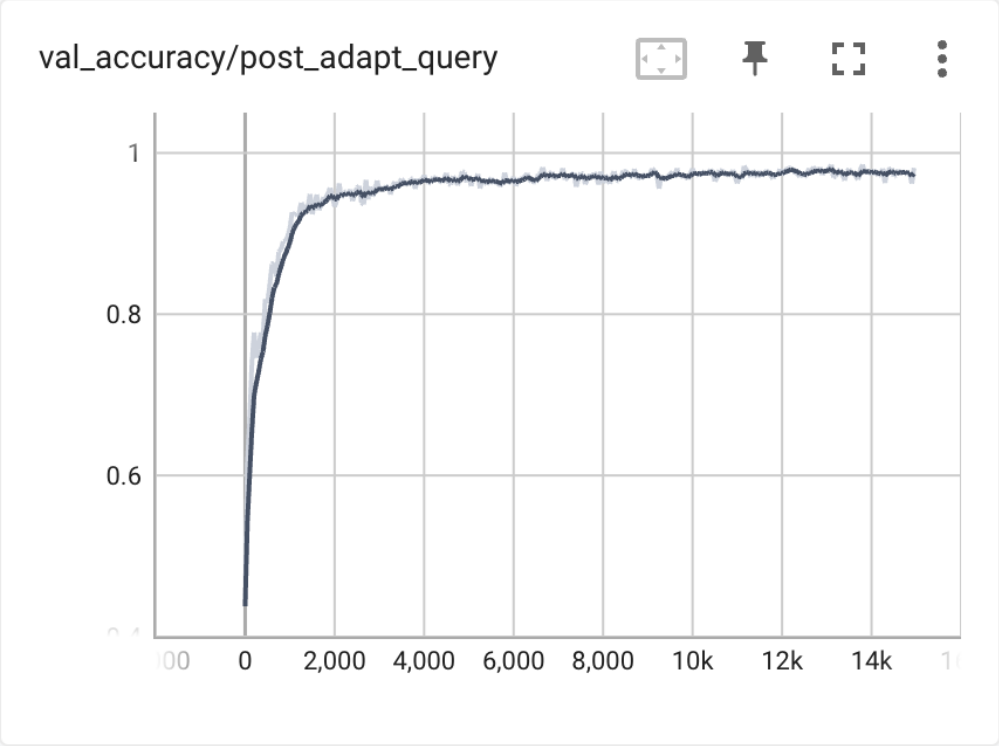


Figure 2b: MAML implementation. Smoothing was set to 75% to ease synoptic interpretation.

2.c [5 points (Written)] Accuracy Metrics

(i) State and explain the behavior of the train_pre_adapt_support and val_pre_adapt_support accuracies. Your answer should explicitly refer to the task sampling process. Hint: consult the omniglot.py file.

Answer:
Both the *training* (left) and *validation* (right) splits demonstrate an expected random variation about a mean of 20% in their *pre-adaptation* accuracies (Figure 2c.1 below). This is to be expected given that we are doing 5-way (N=5) classification with uniform shuffling (i.e. 1 in 5 chance or 20% accuracy) as implemented in the omniglot.py OmniglotSampler() class, __get_item__() function. Specifically, "data for each class is sampled uniformly at random without replacement" (using numpy random.default_rng()).

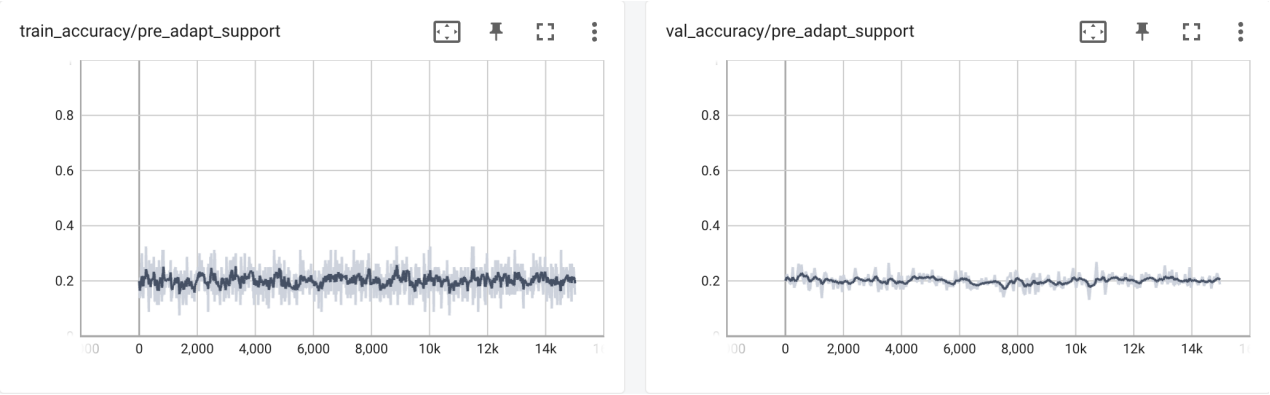


Figure 2c.i: Accuracies for Pre-Adaptation (left) and Post-Adaptation (right) on the Support training split. Smoothing was set to 75% to ease synoptic interpretation.

(ii) Compare the train_pre_adapt_support and train_post_adapt_support accuracies. What does this comparison tell you about the model? Repeat for the corresponding val accuracies.

Answers:

train_pre_adapt_support and train_post_adapt_support accuracies:

We illustrated in the previous answer why a 20% accuracy is to be expected *pre-adaptation*. *Post-adaptation* accuracy (right plot) quickly converges to near unity (100%) in the *training* split shown in the figure below. This is afforded by the MAML architecture and specifically inner loop adaptation mechanism which optimizes the 4 layered CNN network parameters using the Support data while in training and in evaluation alike.

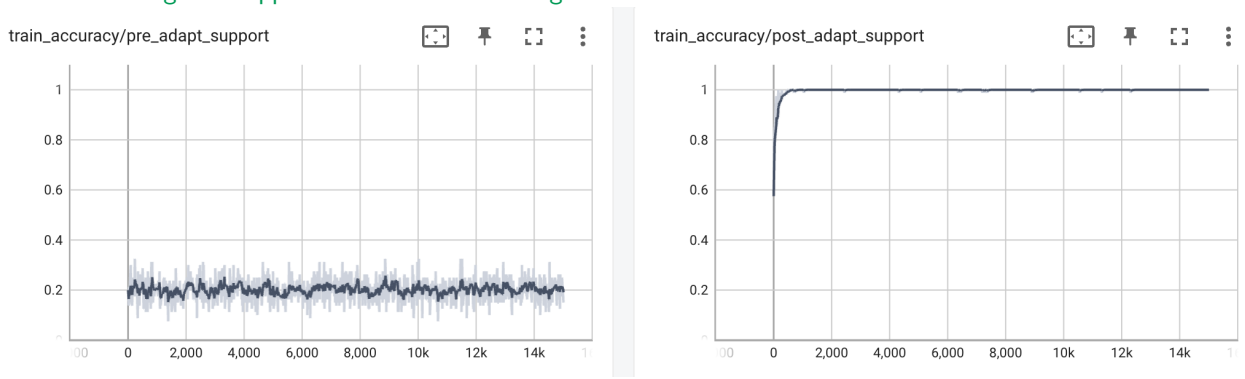


Figure 2c.ii.A: Accuracies on the **training** split pre-adaptation (left) and post-adaptation (right).

val_pre_adapt_support and val_post_adapt_support accuracies:

For the same inner loop adaptation reason given above, we see the same *post-adaptation* performance on the *validation* split (right plot below).

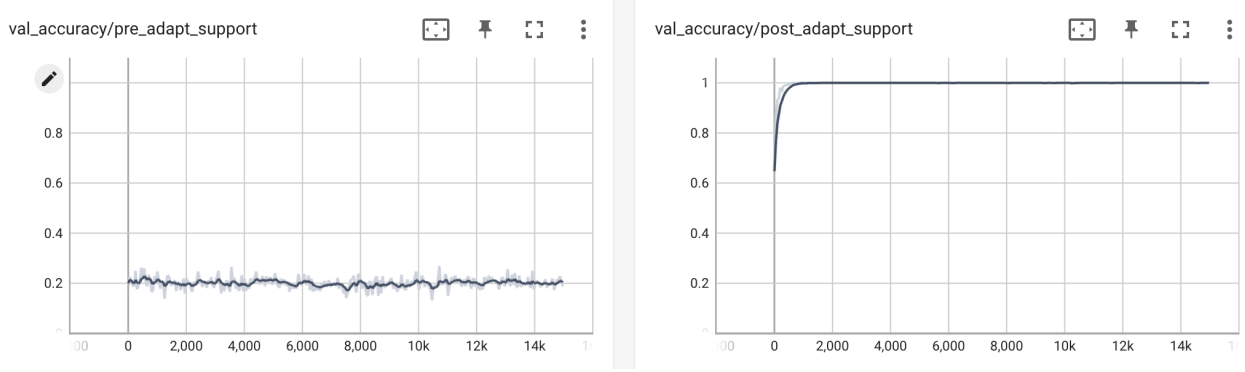


Figure 2c.ii.B: Accuracies on the **validation** split pre-adaptation (left) and post-adaptation (right).

(iii) Compare the train_post_adapt_support and train_post_adapt_query accuracies. What does this comparison tell you about the model? Repeat for the corresponding val accuracies.

Answer:

train_post_adapt_support and train_post_adapt_query:

Overfitting: Zooming in on the *post-adaptation* accuracy figures for the *training* split, we see extremely good performance, i.e. near unity accuracy (100%) on the training split (left plot) and 98% on the Support data (left plot, final iterations) and Query data (right plot) in Figure 2c.iii.A. What does this tell us about the model? Clearly the model is overfitting the Support data to a degree. (y-axis range of [0.8, 1.1] is shown).

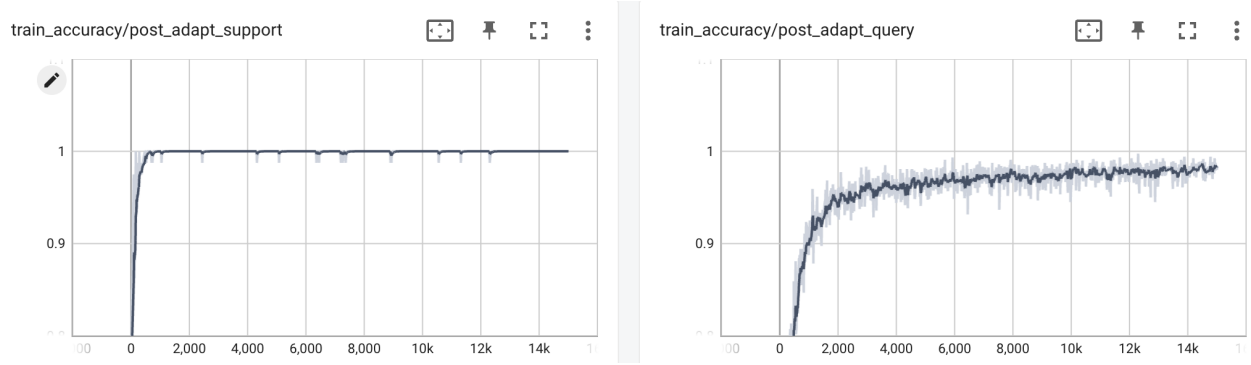


Figure 2c.iii.A: Support (left) and Query (right) data *post-adaptation* accuracies on the **training** split.

val_post_adapt_support and val_post_adapt_query:

Similarily, for the *validation* split, we see near unity accuracy (100%) on the Support data (left plot) and a slightly reduced 97% on the Query data (right plot, final iterations) in Figure 2c.iii.B. While Query data performances are good, slight model overfitting is demonstrated in both the training and validation split accuracies.

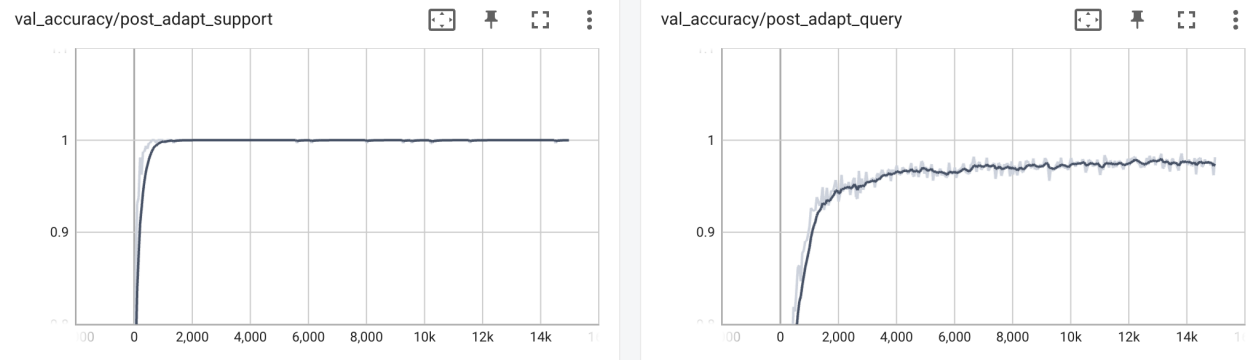


Figure 2c.iii.B: Support (left) and Query (right) data *post-adaptation* accuracies on the **validation** split.

2.d [5 points (Written)] Experiments - Adjusting Learning Rate and Inner Loop Steps

(i) Try MAML with the same hyperparameters as above except for a fixed inner learning rate of 0.04 by running "python maml.py –inner lr 0.04". Your plot of the validation post-adaptation query accuracy over the course of training with the two inner learning rates (0.04, 0.4) should look as in the Handout.

What is the effect of lowering the inner learning rate on (outer-loop) optimization and generalization?

Answer:
The effect of lowering the inner learning rate by a factor of 10 (from 0.4 to 0.04) slows down (more iterations) converging toward the optimal solution, and does not increase generalizability, i.e. does not increase the validation post-adaptation accuracy on the Query data set. This is not really surprising as we have adjusted (lowered) the global learning rate for all model parameters. We need a parameter specific learning rate, as we'll see in (iii) below.

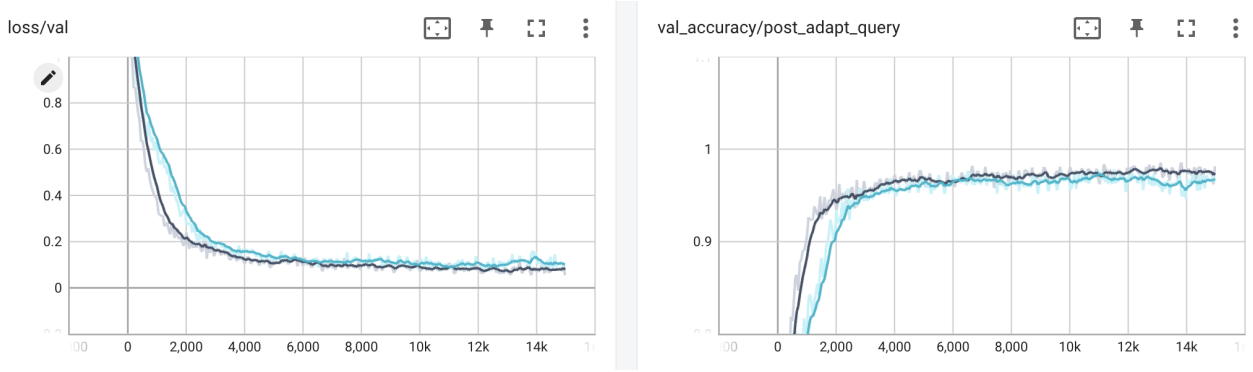


Figure 2d.i: Validation loss (left) and validation accuracy (right) of the post-adaptation Query dataset. Learning rate of 0.4 (default) is colored black, and 0.04 (default/10) is colored blue. Both curves are 5-way, 1-shot, 15-query, 1-inner step, and fixed inner step learning rates.
Black: omniglot.way_5.support_1.query_15.inner_steps_1.inner_lr_0.4.learn_inner_lrs_False.outer_lr_0.001.batch_size_16
Blue: omniglot.way_5.support_1.query_15.inner_steps_1.inner_lr_0.04.learn_inner_lrs_False.outer_lr_0.001.batch_size_16.

(ii) Try MAML with a fixed inner learning rate of 0.04 for 5 inner loop steps by "python maml.py –inner lr 0.04 —-num_inner steps 5". Your plot of the validation post-adaptation query accuracy over the course of training with the two number of inner loop steps (1, 5) with inner learning rate 0.04 should look as in the Handout.

What is the effect of increasing the number of inner loop steps on (outer-loop) optimization and generalization?

Answer:
Increasing the number of inner loop steps negatively impacts the early iterations performance of our model, at least for the earlier several thousand iterations. The end result is roughly a similarly performing model (both optimization and generalization) at the final iteration, with poorer performance in the 5-inner step case (red) for the iterations leading up to near the final 15,000th iteration. There is an interesting semi-periodic sawtooth feature with roughly a 2500-3000 iteration period–probably worth further exploration. Finally, note that our implementation results shown here are visually consistent with the predicted performance shown in the Handout.

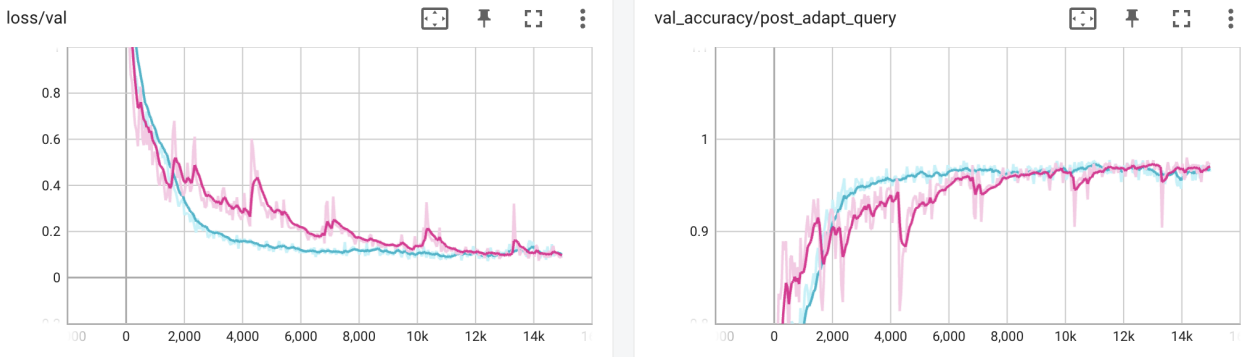


Figure 2d.ii: Validation loss (left) and post-adaptation Query data accuracy (right) for the case of 1 inner loop step (blue traces) and 5 inner loop steps (red traces). Validation loss and post-adaptation Query data accuracy were chosen as they are good measures of final outer loop optimization, and task generalization, respectively. All traces are for a fixed inner loop learning rate of 0.04.

(iii) Try MAML with learning the inner learning rates by running "python maml.py –learn_inner lrs". Initialize the inner learning rates with 0.4. Your plot of the validation post-adaptation query accuracy over the course of training for learning and not learning the inner learning rates, initialized at 0.4 should look as follows..

What is the effect of learning the inner learning rates on (outer-loop) optimization and generalization?

Answer:
The effect of learning the inner learning rates is achieving an optimal solution (left figure) and good general accuracy (right) more quickly–lower number of required iterations. Visually, the performances in terms of validation loss and post-adaptation outer loop accuracy are not different within statistical significance. Furthermore, running "python maml.py –test" corroborates this last statement:

Case: Not Learning the Inner Learning Rate:

```
$ python maml.py --test --num_way 5 --num_support 1 --checkpoint_step 14900 --device gpu
Namespace(log_dir=None, num_way=5, num_support=1, num_query=15, num_inner_steps=1, inner_lr=0.4, learn_inner_lrs=False, outer_lr=0.0
Using device:  cuda
log_dir: ./logs/maml/omniglot.way_5.support_1.query_15.inner_steps_1.inner_lr_0.4.learn_inner_lrs_False.outer_lr_0.001.batch_size_16
Loaded checkpoint iteration 14900.
Testing on tasks with composition num_way=5, num_support=1, num_query=15
```

Accuracy over 600 test tasks: mean 0.977, 95% confidence interval 0.003

Case: Yes, Learning the Inner Learning Rate:

```
$ python maml.py --test --num_way 5 --num_support 1 --checkpoint_step 14900 --learn_inner_lrs --device gpu
Namespace(log_dir=None, num_way=5, num_support=1, num_query=15, num_inner_steps=1, inner_lr=0.4, learn_inner_lrs=True, outer_lr=0.00
Using device:  cuda
log_dir: ./logs/maml/omniglot.way_5.support_1.query_15.inner_steps_1.inner_lr_0.4.learn_inner_lrs_True.outer_lr_0.001.batch_size_16
Loaded checkpoint iteration 14900.
Testing on tasks with composition num_way=5, num_support=1, num_query=15
```

Accuracy over 600 test tasks: mean 0.979, 95% confidence interval 0.003

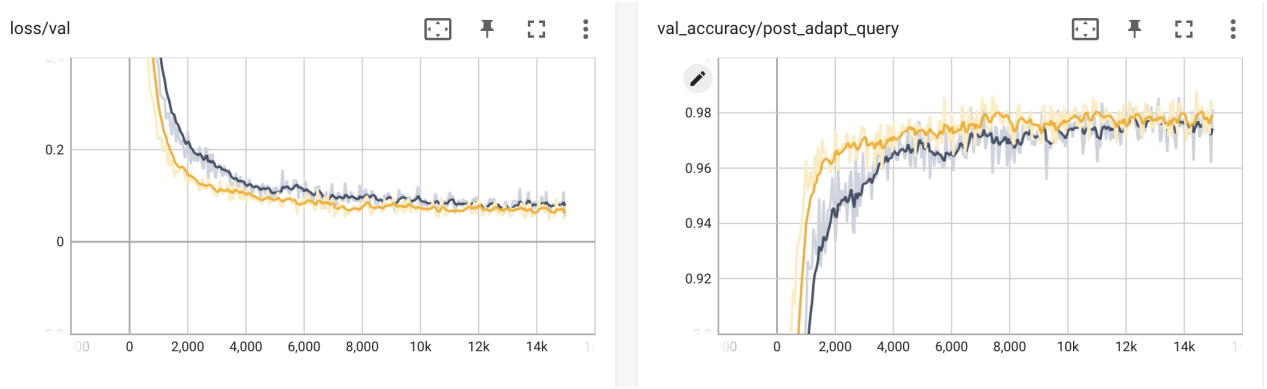


Figure 2d.iii: Validation loss (left) and post-adaptation Query data accuracy (right) for the case of NOT learning the inner learning rates (black traces) and YES learning the inner learning rates (yellow traces). Validation loss and post-adaptation Query data accuracy were chosen as they are good measures of final outer loop optimization, and task generalization, respectively. Notes, compared to prior figures, we've zoomed in the y-axes a touch more to take an even closer look.